



UNIVERSITÀ DEGLI STUDI DI NAPOLI
FEDERICO II

DOCUMENTO DI DESIGN DEL SOFTWARE "BUGBOARD26"

CORSO DI INGEGNERIA DEL SOFTWARE

Gabriele Letizia - N86005267

Fabio Iannicelli - N86005383

ANNO ACCADEMICO 2025/26

Indice

1	Introduzione	3
2	Descrizione dello schema per la persistenza dati	4
3	Design del software	6
3.1	Diagramma delle classi di design	6
3.2	Diagrammi comportamentali	6
3.2.1	Statechart Diagram	6
3.2.2	Activity Diagram	7
3.3	Descrizione e motivazione delle scelte di software design	8
3.3.1	Architettura Multi-Tier e Pattern MVC	8
3.3.2	Data Transfer Object (DTO) Pattern	8
3.3.3	Repository Pattern	8
3.3.4	Singleton Pattern: Gestione della Sessione e IoC	8
3.3.5	Builder Pattern	9
3.3.6	Observer Pattern e Collezioni Osservabili	9
3.3.7	Architettura REST e Statelessness	9
3.3.8	Gestione delle Password	10
3.3.9	Gestione della Configurazione	10
3.3.10	Gestione della Sicurezza e delle Chiavi Crittografiche	10
4	Analisi della Qualità	11
4.1	Report di Qualità del Codice (SonarQube)	11
5	Processo di sviluppo e versioning	13
5.1	Gestione del codice sorgente e versioning	13
6	Codice Sorgente, Artefatti e Deployment	15
6.1	Build Automation (Maven)	15
6.2	Containerizzazione del Back-end (Dockerfile)	15
6.3	Orchestrazione e Rilascio in Cloud (Docker Compose e AWS EC2)	15

1 Introduzione

Il seguente documento descrive scelte architetturali, metodologie di design e artefatti software prodotti durante lo sviluppo di **BugBoard26**, un'applicazione client-server dedicata alla gestione collaborativa di issue in progetti software.

Il progetto è stato strutturato adottando un'architettura distribuita e cloud-ready, che separa nettamente la logica di presentazione (Front-end) dalla logica di business e persistenza dei dati (Back-end).

Il sistema si compone di un'interfaccia client desktop sviluppata in JavaFX e di un server RESTful basato su Spring Boot, interfacciato a un database relazionale PostgreSQL. Tramite Docker è stato possibile containerizzare l'infrastruttura di Back-end e distribuirla su un'istanza cloud **Amazon Web Services (AWS) EC2**, favorendo l'isolamento dell'ambiente di esecuzione, la riproducibilità della configurazione e la semplicità del deployment.

L'intero processo di sviluppo è stato supportato da pratiche moderne di Ingegneria del Software, tra cui il controllo di versione distribuito e l'analisi statica del codice, ciò ha favorito una maggior manutenibilità e qualità del codice.

Il documento è strutturato nelle seguenti sezioni:

- **Persistenza dei Dati:** Descrizione dello schema relazionale e delle strategie di memorizzazione adottate per il database PostgreSQL.
- **Design del Software:** Presentazione del diagramma delle classi di design, accompagnato dall'analisi e dalla motivazione dei design pattern e delle scelte architetturali adottate per risolvere specifici problemi di progettazione.
- **Processo di Sviluppo e Versioning:** Tracciamento delle attività del team tramite le metriche del repository GitHub.
- **Analisi della Qualità:** Esposizione dei report di ispezione del codice generati tramite strumenti di analisi statica (SonarQube) per il Back-end, evidenziando le principali metriche relative ad affidabilità, sicurezza e manutenibilità dell'implementazione.
- **Artefatti e Deployment:** Riferimenti al codice sorgente e alla configurazione degli strumenti di build automation (Maven) e containerizzazione (Dockerfile e docker-compose.yml) necessari per il rilascio del software.

2 Descrizione dello schema per la persistenza dati

Il livello di persistenza dei dati è stato progettato secondo il paradigma relazionale ed implementato su PostgreSQL. Lo schema fisico riflette in modo diretto il mapping Object-Relational delle entità di dominio definite nel Back-end (Spring Data JPA), garantendo consistenza tra la logica a oggetti e lo storage relazionale.

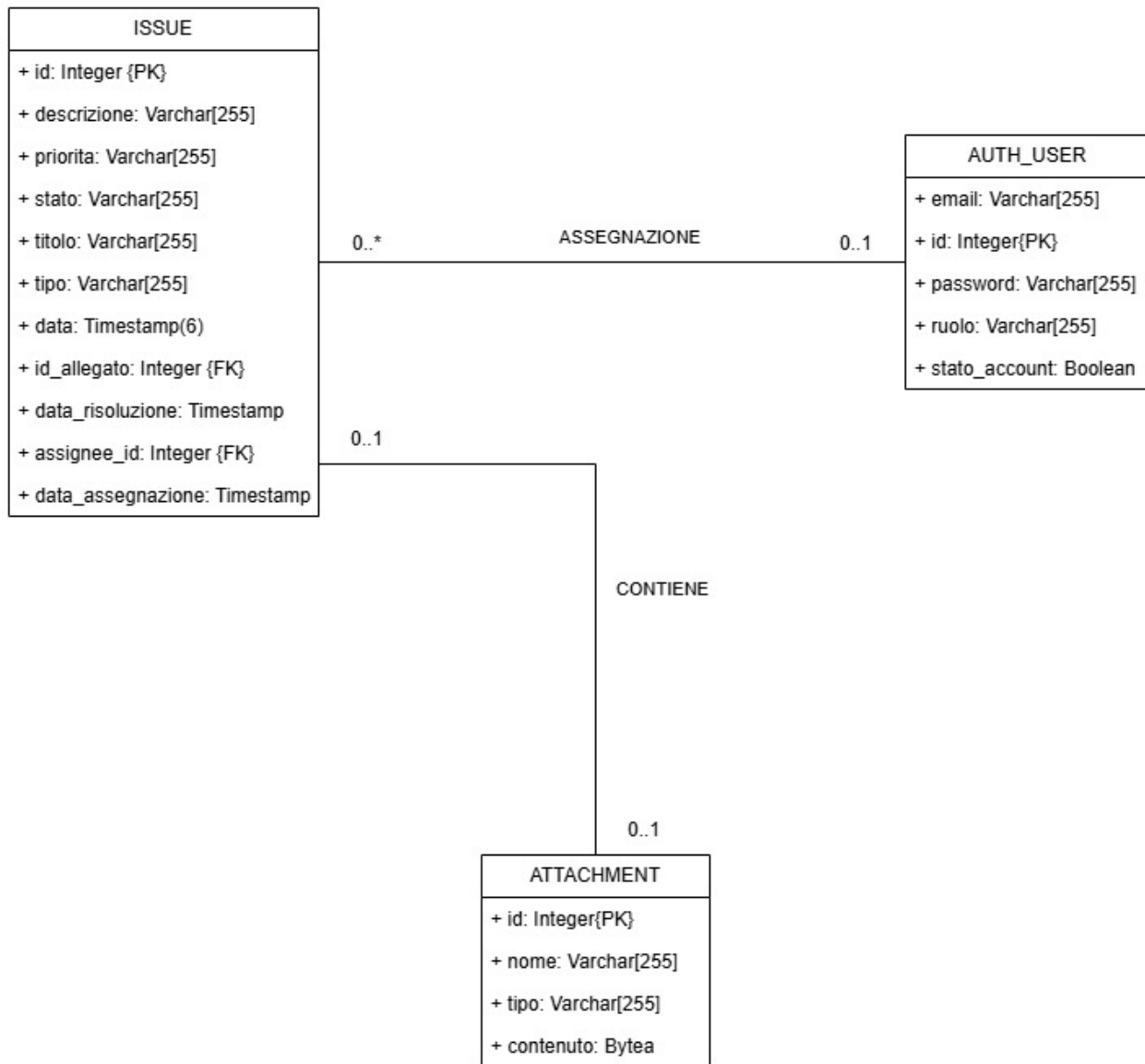


Figura 1: Schema relazionale del database di BugBoard26.

Le scelte architettureali e di design del database si articolano sui seguenti punti:

1. **Scelta delle Chiavi Primarie:** Per tutte le tabelle del sistema (auth_user, issue, attachment) si è optato per l'utilizzo di chiavi primarie surrogate (id di tipo Integer autoincrementante). Questa scelta disaccoppia l'identità fisica del record dalle informazioni di business, garantendo che eventuali modifiche non intacchino l'integrità referenziale delle chiavi esterne.
2. **Mappatura dei tipi di dato e degli Enumeratori:** I tipi di dato fisici sono stati scelti in funzione delle caratteristiche dei dati da rappresentare e della leggibilità dello schema:
 - **Enumeratori come Stringhe:** I campi che a livello applicativo sono gestiti tramite Enum (come ruolo, stato e tipo della Issue) sono stati mappati sul database come VARCHAR(255). Questa scelta garantisce una leggibilità immediata dei record eseguendo query dirette sul database e rende il sistema flessibile a futuri inserimenti di nuovi stati evitando la necessità di modificare tipi enumerati definiti direttamente a livello di database.
 - **Persistenza dei File:** Per la gestione degli allegati si è scelto di salvare il file direttamente all'interno del database nella tabella attachment, utilizzando il tipo BYTEA. Considerando che il sistema prevede allegati leggeri, questa scelta centralizza il backup e semplifica il deployment, evitando la necessità di configurare un file system esterno o un bucket cloud.
 - **Gestione Date:** I campi relativi alle date di transizione di stato delle issue (data, data_risoluzione, data_assegnazione) sono stati mappati mediante il tipo TIMESTAMP. Questo permette tracciabilità esatta e calcolo preciso delle metriche operative.
3. **Implementazione delle Relazioni e Vincoli di Integrità referenziale:** Le associazioni tra le entità sono state modellate per rispettare le cardinalità del dominio, demandando al database il controllo sull'integrità referenziale:
 - **Relazione ASSEGNAZIONE (Issue ↔ Auth_User):** Modellata tramite la Foreign Key assignee_id all'interno della tabella issue. Questo vincolo permette a una specifica issue di avere al massimo un solo utente assegnatario (0..1), mentre lo stesso utente può prendere in carico un numero non specificato di segnalazioni (0..*).
 - **Relazione CONTIENE (Issue ↔ Attachment):** Modellata tramite la Foreign Key id_allegato all'interno della tabella issue. Per imporre la cardinalità Uno-a-Uno ed evitare che uno stesso allegato venga associato per errore a più issue diverse, oltre al vincolo referenziale è stato applicato un vincolo di univocità (UNIQUE) sulla colonna id_allegato.

3 Design del software

3.1 Diagramma delle classi di design

Il Diagramma delle Classi di Design illustra la struttura del sistema, evidenziando le classi principali, i loro attributi, i metodi e le relazioni tra di esse. Per gestire la complessità e favorire la separazione delle responsabilità, il sistema è stato logicamente suddiviso in due macro-componenti: il **Client** (Front-end JavaFX) e il **Server** (Back-end Spring Boot).

Visto l'elevato numero di classi, i due diagrammi non sono stati inclusi direttamente in questo documento, ma sono visualizzabili all'interno della cartella "**Class Diagram**", presente nella directory "**Documentazione Finale**" del materiale consegnato.

3.2 Diagrammi comportamentali

3.2.1 Statechart Diagram

Per rappresentare il comportamento della singola issue durante il suo ciclo di vita è stato realizzato uno Statechart Diagram. Il diagramma mostra le transizioni tra gli stati della issue in funzione delle operazioni effettuate dal sistema, dalla creazione fino all'assegnazione, alla risoluzione, all'archiviazione o all'eliminazione.

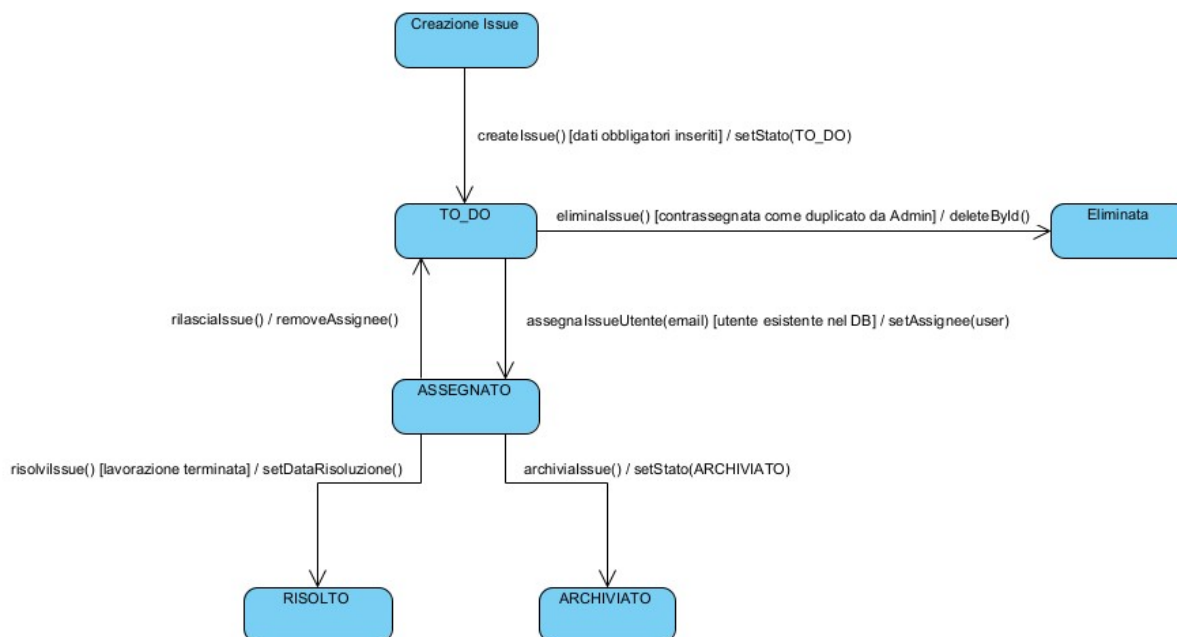


Figura 2: Statechart Diagram relativo al ciclo di vita di una issue.

3.2.2 Activity Diagram

Per la modellazione del comportamento dinamico del sistema è stato realizzato un Activity Diagram relativo al processo di assegnazione e lavorazione di una issue. Il diagramma descrive il flusso seguito a partire dalla richiesta di assegnazione, verificando la validità dei dati forniti e distinguendo successivamente tra la risoluzione della issue e il suo rilascio o la condizione di non risolvibilità.

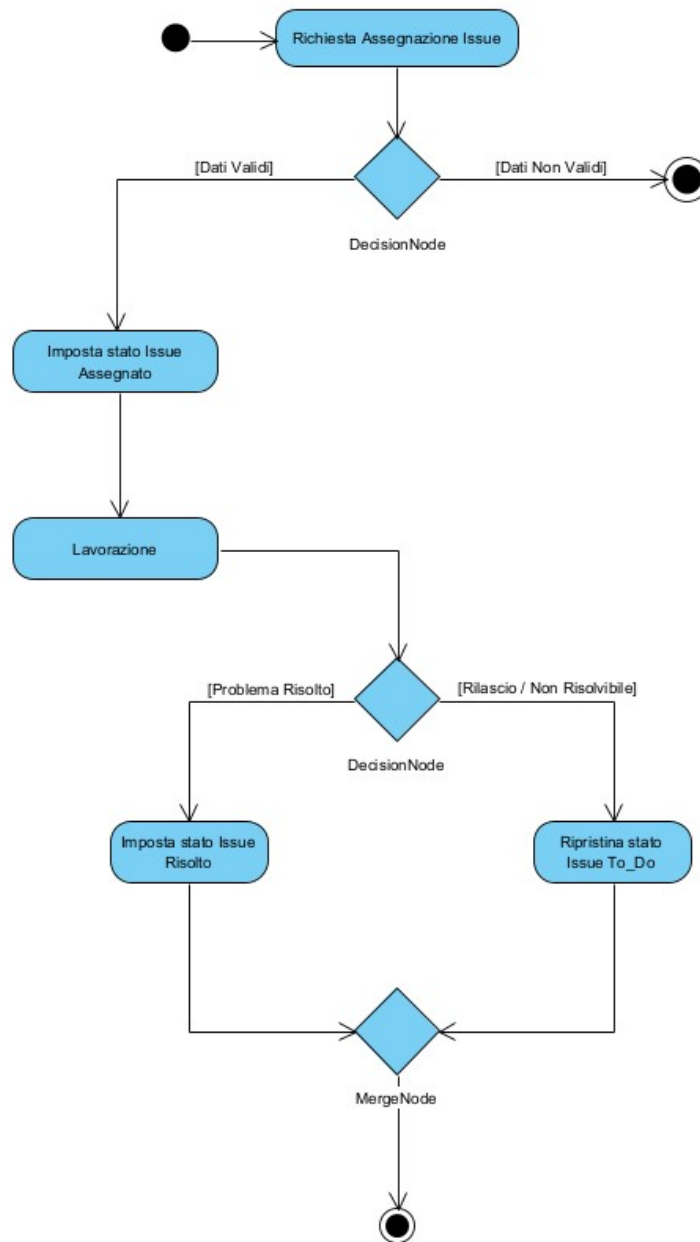


Figura 3: Activity Diagram relativo al processo di assegnazione e lavorazione di una issue.

3.3 Descrizione e motivazione delle scelte di software design

La progettazione architetturale e di dettaglio del sistema è stata guidata dall'obiettivo di massimizzare la coesione, ridurre l'accoppiamento e favorire la manutenibilità del codice. Sono stati pertanto adottati diversi design pattern e stili architetturali standard.

3.3.1 Architettura Multi-Tier e Pattern MVC

L'intero sistema adotta un'architettura Client-Server distribuita. I due moduli sono organizzati secondo una separazione in livelli, con un'applicazione del pattern MVC particolarmente evidente nel Front-end JavaFX e una struttura a strati nel Back-end Spring Boot:

- **Front-end (Client):** Le interfacce grafiche (View) sono definite in modo dichiarativo tramite file FXML, separando nettamente il design dalla logica di interazione, che è demandata ai Controller JavaFX. La comunicazione di rete e le funzionalità di supporto sono state incapsulate in apposite classi di servizio e Helper, tra cui `ApiClient`.
- **Back-end (Server):** Segue una struttura a strati. Le richieste HTTP in ingresso vengono gestite dai `RestController` (Controller), che ne validano il formato e delegano la logica di business ai `Service`, i quali a loro volta interagiscono con il livello di persistenza tramite i `Repository`.

3.3.2 Data Transfer Object (DTO) Pattern

Per la comunicazione tra Front-end e Back-end sono stati utilizzati i Data Transfer Object (DTO), evitando di esporre direttamente le entità JPA attraverso le API REST. I DTO contengono esclusivamente le informazioni necessarie alle singole operazioni, consentendo di controllare esplicitamente i dati esposti dalle API. Questa scelta riduce l'accoppiamento tra il livello di persistenza e l'interfaccia applicativa, evitando inoltre di rendere direttamente accessibili tutti gli attributi delle entità persistenti e prevenendo problematiche legate alla serializzazione delle relazioni tra entità.

3.3.3 Repository Pattern

Per l'accesso ai dati persistenti è stato adottato il pattern **Repository**, implementato tramite il framework *Spring Data JPA*. I Repository forniscono un'astrazione sulle operazioni di persistenza, incluse le operazioni CRUD e le query necessarie all'applicazione, riducendo l'accoppiamento tra la logica applicativa e il livello di accesso ai dati. In questo modo, la gestione della persistenza risulta centralizzata e maggiormente manutenibile.

3.3.4 Singleton Pattern: Gestione della Sessione e IoC

Il pattern Singleton è stato sfruttato attivamente in entrambi i moduli del sistema, ma con due approcci differenti:

- **Lato Client (Manuale):** È stata implementata una classe `AuthSession` come Singleton canonico. Questa scelta garantisce un unico punto di accesso globale per il salvataggio dello stato di autenticazione dell'utente (es. dati utente e token), rendendolo facilmente accessibile da qualsiasi controller grafico dell'applicazione senza dover passare l'oggetto di finestra in finestra.

- **Lato Server (Inversion of Control):** Spring gestisce il ciclo di vita dei componenti tramite il proprio container IoC. I bean, tra cui Controller, Service e Repository, utilizzano di default lo scope Singleton e vengono forniti alle classi che ne dipendono tramite Dependency Injection. Questo approccio riduce l'accoppiamento tra i componenti e ne facilita la sostituibilità e il testing.

3.3.5 Builder Pattern

Per la configurazione del client HTTP nel Front-end, utilizzato per effettuare le chiamate REST verso il Back-end, deployato su infrastruttura AWS, è stato utilizzato il **Builder Pattern**. Sfruttando le API native di Java (`HttpClient.newBuilder()`), è stato possibile configurare in modo pulito e sequenziale parametri complessi, come i timeout di connessione, ottenendo infine un'istanza configurata e riutilizzabile del client HTTP.

3.3.6 Observer Pattern e Collezioni Osservabili

L'interfaccia utente in JavaFX sfrutta il meccanismo Observer attraverso le collezioni osservabili. Le liste utilizzate dai componenti grafici, come le tabelle delle issue, sono collegate ai dati sottostanti tramite *ObservableList*. Le modifiche apportate ai dati in memoria vengono automaticamente notificate ai componenti grafici interessati, che possono aggiornare la propria rappresentazione senza richiedere procedure manuali di refresh, garantendo un'interfaccia reattiva.

3.3.7 Architettura REST e Statelessness

Il livello di comunicazione tra client e server segue i principali vincoli dell'architettura REST, tra cui il principio di statelessness. Il Back-end non mantiene uno stato di sessione associato al singolo client tra una richiesta e l'altra.

Questa scelta consente al server di gestire ogni richiesta in maniera indipendente dalle richieste precedenti, mantenendo separato lo stato della sessione dal livello applicativo.

L'adozione di un'architettura stateless risulta inoltre coerente con il successivo deployment del Back-end su infrastruttura cloud AWS, poiché non richiede la persistenza di uno stato di sessione lato server.

3.3.8 Gestione delle Password

Per proteggere le credenziali degli utenti, il sistema evita la memorizzazione delle password in chiaro all'interno del database. Per la memorizzazione sicura delle credenziali è stato scelto l'algoritmo di hashing **BCrypt**, la cui robustezza si basa principalmente su due meccanismi:

- **Generazione automatica del Salt:** BCrypt utilizza un valore casuale, denominato *salt*, per ogni password prima di calcolarne l'hash. In questo modo, due utenti con la stessa password produrranno hash differenti nel database, rendendo inefficaci gli attacchi basati su *rainbow tables*, ovvero tabelle pre-calcolate contenenti corrispondenze tra password e relativi hash.
- **Work Factor configurabile:** L'algoritmo è progettato per rendere computazionalmente oneroso il calcolo dell'hash. Il parametro *work factor* consente di configurare il costo computazionale dell'operazione, aumentando di conseguenza il tempo necessario per effettuare tentativi massivi di password e rendendo più onerosi gli attacchi di forza bruta.

3.3.9 Gestione della Configurazione

Per separare i parametri di configurazione dal codice sorgente, le credenziali e i parametri relativi alla connessione al database sono gestiti mediante variabili d'ambiente. In particolare, nell'ambiente di deployment containerizzato viene utilizzato un file `.env`, impiegato da Docker Compose per fornire al database e al Back-end i parametri di configurazione necessari. Il file `.env` non viene incluso nel versionamento Git, così da evitare l'esposizione delle credenziali e degli altri parametri sensibili.

3.3.10 Gestione della Sicurezza e delle Chiavi Crittografiche

Autenticazione: L'autenticazione del sistema è basata su JWT ed è integrata mediante Spring Security. La configurazione relativa alla sicurezza è centralizzata nelle apposite classi del package `config`, che definiscono le modalità di gestione delle richieste HTTP e di validazione dei token JWT, mantenendo separata la configurazione della sicurezza dalla logica applicativa.

Chiavi: Le chiavi crittografiche RSA utilizzate per la gestione dei token JWT sono organizzate all'interno dell'apposita directory `Keys`. La chiave privata viene utilizzata per la firma dei token, mentre la chiave pubblica viene utilizzata per la relativa verifica.

I file contenenti le chiavi crittografiche sono esclusi dal versionamento Git, al fine di evitare l'esposizione di informazioni sensibili.

4 Analisi della Qualità

Per garantire la manutenibilità, l'affidabilità e la sicurezza del sistema, il codice sorgente del Back-end è stato sottoposto ad analisi statica automatizzata mediante SonarQube. Tale analisi è stata integrata nel processo di build tramite Maven, consentendo di effettuare una valutazione complessiva del progetto e di rilevare eventuali problematiche relative alla qualità del codice.

4.1 Report di Qualità del Codice (SonarQube)

Di seguito sono riportate le principali metriche globali rilevate durante l'ultima scansione eseguita, che evidenziano il superamento del Quality Gate configurato secondo le impostazioni predefinite di SonarQube.

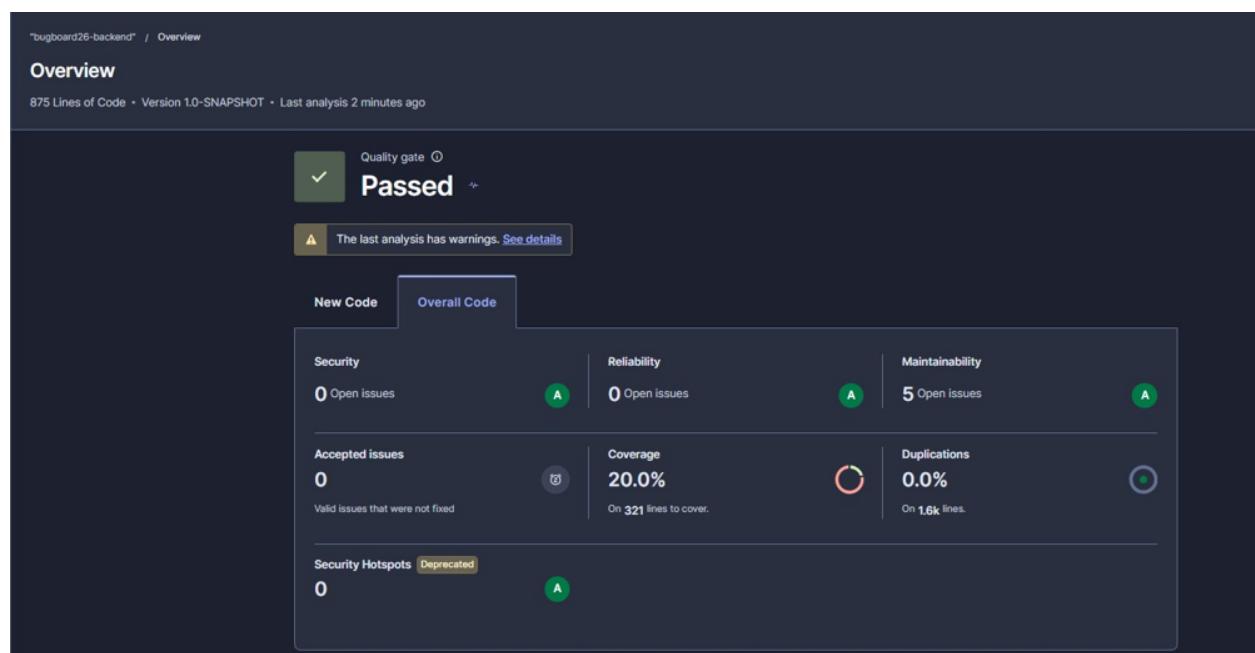


Figura 4: Risultati dell'analisi statica del Back-end mediante SonarQube.

Analisi dei risultati: L'analisi eseguita ha restituito un esito positivo, con il Quality Gate superato. Le principali metriche rilevate sono riportate di seguito.

- **Bugs (Reliability): 0** — non sono state rilevate problematiche classificate da SonarQube come bug, ovvero anomalie del codice potenzialmente in grado di provocare comportamenti errati durante l'esecuzione.
- **Vulnerabilities (Security): 0** — non sono state individuate vulnerabilità di sicurezza nel codice analizzato.
- **Security Hotspots: 0** — non sono presenti porzioni di codice che richiedano una revisione manuale specifica sotto il profilo della sicurezza.

- **Code Smells (Maintainability): 5** — sono state rilevate cinque problematiche relative alla manutenibilità del codice. I Code Smells non rappresentano necessariamente errori funzionali o vulnerabilità, ma indicano costrutti che potrebbero essere migliorati per rendere il codice più leggibile, semplice da modificare e mantenere. Il debito tecnico associato a tali segnalazioni è stimato da SonarQube in 30 minuti.
- **Coverage: 20%** — indica la percentuale di codice eseguito dai test automatici rispetto al codice individuato da SonarQube come sottoponibile a copertura. Nel caso analizzato, la copertura complessiva è pari al 20% delle 321 linee individuate come soggette a copertura.
- **Duplications: 0,0%** — non sono state rilevate porzioni di codice duplicate secondo i criteri adottati da SonarQube, evidenziando l'assenza di duplicazioni significative nel codice analizzato.

Nel complesso, l'analisi evidenzia l'assenza di problematiche relative a bug, vulnerabilità e security hotspot. Sono invece presenti cinque segnalazioni di tipo Code Smell, associate a un debito tecnico stimato contenuto e quantificato da SonarQube in 30 minuti. Il valore di duplicazione pari allo 0,0% costituisce inoltre un indicatore positivo sotto il profilo della riduzione del codice ridondante. La coverage del 20% evidenzia invece che la suite di test automatizzati copre una parte limitata del codice analizzato; tale metrica riguarda specificamente l'estensione dei test e non costituisce, di per sé, una misura della correttezza o della qualità complessiva dell'applicazione.

5 Processo di sviluppo e versioning

5.1 Gestione del codice sorgente e versioning

Per la gestione del codice sorgente, il tracciamento delle modifiche e la collaborazione tra i membri del team è stato adottato **Git** come sistema di controllo di versione distribuito, utilizzando **GitHub** come piattaforma per l'hosting del repository remoto.

L'adozione di un sistema di versioning ha consentito di mantenere uno storico completo e affidabile delle modifiche apportate al progetto, facilitando il coordinamento tra gli sviluppatori e l'integrazione delle diverse componenti del sistema. Lo sviluppo delle funzionalità è stato inoltre organizzato mediante l'utilizzo di **branch dedicati**, permettendo di isolare le modifiche relative alle singole attività e di integrarle successivamente nel ramo principale.

Dettagli del repository:

- **URL del repository:** <https://github.com/galetizia/Progetto-INGSW>
- **Composizione del codice:** Java (65,3%), HTML (30,3%), CSS (4,4%)

Statistiche di sviluppo e contributi: L'analisi delle metriche del repository evidenzia un'attività di sviluppo condivisa tra i membri del team. I dati riportati di seguito sono stati rilevati al termine della fase di sviluppo e sono aggiornati a **settembre 2026**:

- **Contributors:** 2 sviluppatori attivi.
- **Commit complessivi:** circa 200.
- **Periodo di sviluppo:** il tracciamento delle attività copre un arco temporale di circa 11 mesi, da ottobre 2025 a settembre 2026. L'analisi della frequenza dei commit evidenzia due principali fasi di attività, con una significativa intensificazione dello sviluppo e una maggiore frequenza dei rilasci nei mesi di agosto e settembre 2026, in concomitanza con la finalizzazione dell'architettura e della documentazione.

Strategia di collaborazione: Al fine di garantire la stabilità del codice presente nel ramo principale, il team ha organizzato lo sviluppo mediante la suddivisione delle attività su **branch distinti**. Le nuove funzionalità e le modifiche alla configurazione del progetto sono state sviluppate utilizzando branch dedicati e successivamente integrate nel branch principale (**main**) dopo aver verificato la corretta compilazione del progetto.

Questa modalità di lavoro ha consentito di isolare le diverse attività di sviluppo, riducendo il rischio di interferenze tra le modifiche effettuate dai membri del team e contribuendo a mantenere l'integrità e la stabilità del codice presente nel ramo principale. I **merge** effettuati sul repository documentano quindi l'integrazione delle modifiche sviluppate sui branch dedicati all'interno del ramo principale, fornendo evidenza del processo di consolidamento delle diverse funzionalità.

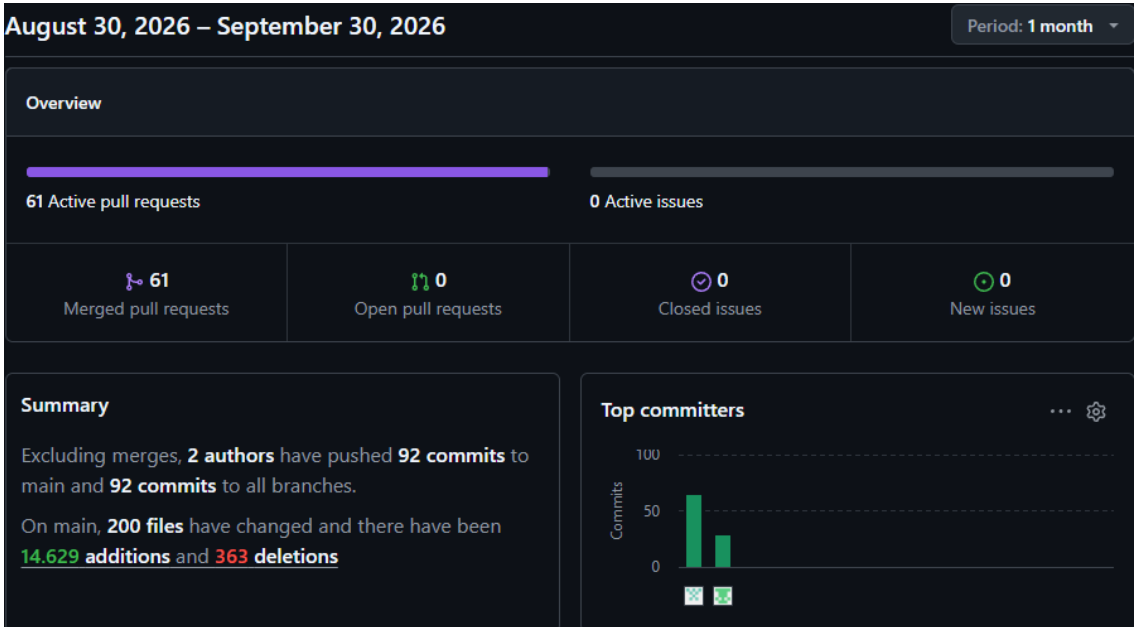


Figura 5: Attività di merge dei branch durante lo sviluppo del progetto.

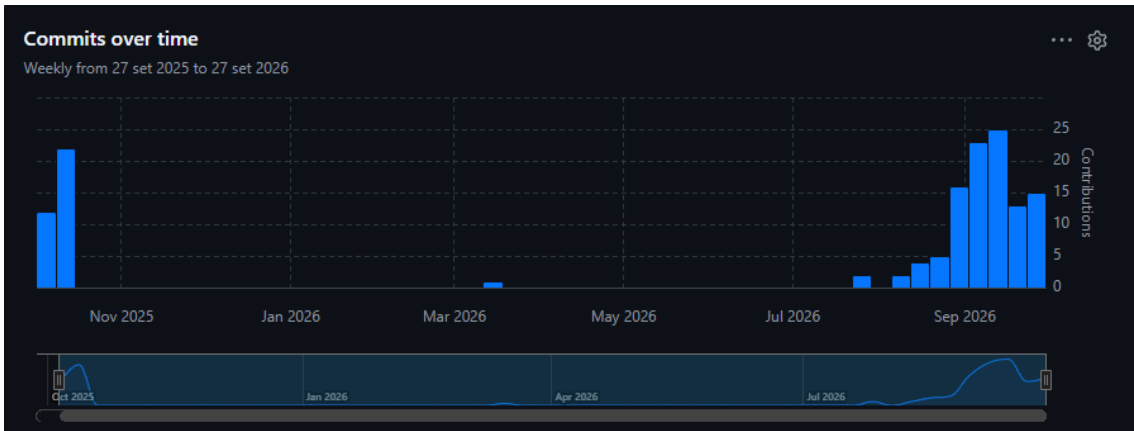


Figura 6: Andamento temporale dei commit del repository.

6 Codice Sorgente, Artefatti e Deployment

Il processo di rilascio del software è stato strutturato al fine di favorire la riproducibilità degli ambienti di esecuzione e semplificare la generazione e la distribuzione degli artefatti. I codici sorgenti dei due moduli, Front-end e Back-end, sono accessibili tramite il repository GitHub citato in precedenza.

6.1 Build Automation (Maven)

Per entrambi i moduli, la gestione delle dipendenze e l'automazione del processo di build sono state affidate a **Apache Maven**. Tramite la configurazione dei rispettivi file `pom.xml`, Maven si occupa di:

- Risolvere e scaricare le librerie necessarie (come moduli JavaFX, driver PostgreSQL, Spring Boot Starter, BCrypt).
- Compilare il codice sorgente secondo la configurazione definita nel progetto.
- Generare gli artefatti di distribuzione, tra cui i file `.jar` prodotti nella cartella `target/`, destinati all'esecuzione o alla successiva containerizzazione del Back-end.

6.2 Containerizzazione del Back-end (Dockerfile)

Al fine di rendere l'ambiente di esecuzione del server maggiormente indipendente dalla configurazione del sistema operativo ospite e favorire la riproducibilità del deployment, l'applicativo Spring Boot è stato containerizzato.

A tale scopo è stato redatto un **Dockerfile** che definisce le istruzioni necessarie alla creazione dell'immagine dell'applicazione. Come immagine di base è stata selezionata ***eclipse-temurin:23-jdk-alpine***, che combina il JDK Eclipse Temurin con una distribuzione Linux minimale basata su Alpine. Tale scelta consente di contenere le dimensioni dell'immagine e di ridurre i componenti non necessari all'esecuzione dell'applicazione.

6.3 Orchestrazione e Rilascio in Cloud (Docker Compose e AWS EC2)

L'intera infrastruttura di Back-end è descritta in modo dichiarativo mediante il file **`docker-compose.yml`**. Docker Compose ne gestisce quindi l'avvio e la configurazione, orchestrando la comunicazione tra i due servizi:

- Il container dell'applicazione **Spring Boot**, la cui immagine viene generata a partire dal Dockerfile.
- Il container del database **PostgreSQL**, inizializzato con le configurazioni necessarie e dotato di volumi persistenti (*volumes*) per preservare i dati anche in caso di ricreazione o rimozione del container.

Grazie a questa configurazione, l'infrastruttura di Back-end è stata rilasciata su un'istanza cloud **Amazon Web Services (AWS) EC2** (classe *t3.micro*). Il client JavaFX, distribuito agli utenti finali come applicazione standalone, è stato configurato per comunicare tramite chiamate REST con l'indirizzo IP pubblico del server AWS, completando l'architettura distribuita del sistema.